Red Wine Quality Prediction

Group: Team Array Ko

Members: Kelly Gabot | Cesar Jondiz III | Areisha Julliana Migrino

Repository: https://github.com/kellygabot/red_wine_quality

Deployed Application: https://kellygabot-red-wine-quality-appapp-a9o7rh.streamlit.app/

Table of Contents

1. Purpose and scope..... 1

2. System requirements and dependencies..... 2

3. Project structure..... 2

4. Installation and local launch..... 3

5. Data and input specification..... 3

    5.1 Dataset files..... 3

    5.2 Application inputs..... 4

6. Application user guide..... 5

    6.1 Select a model..... 5

    6.2 Exploration tab..... 5

    6.3 Model Comparison tab..... 5

    6.4 Prediction tab..... 5

7. Processing and model implementation..... 6

8. Reproducing notebook outputs..... 7

9. Screenshots and verification record..... 7

10. Troubleshooting..... 8

11. Limitations and remaining submission work..... 9

12. Reference material..... 9

1. Purpose and Scope

This document explains how to install, run, reproduce, and use the Red Wine Quality application. It also describes its inputs, outputs, implementation, and known limitations. The application estimates a numerical red-wine quality score from 11 physicochemical measurements. It supports three regression models: Random Forest, XGBoost, and Support Vector Regression (SVR). Users can explore the dataset, compare five-fold validation metrics, and submit measurements for a prediction. The IMRaD paper separately explains the research problem, related literature, experimental justification, results, and interpretation. This technical guide focuses on operating and maintaining the implemented system.

2. System Requirements and Dependencies

Prerequisites

- Python 3.14 or newer.
- uv environment manager: https://docs.astral.sh/uv/.
- A browser and the complete project, including the source CSV and uv.lock.
- Internet access for the initial installation.

Software dependencies

Dependency	Purpose
Streamlit	Application interface, forms, and caching
pandas and NumPy	Data preparation and numerical calculations
scikit-learn	Random Forest, SVR, scaling, splits, and evaluation
XGBoost	Gradient-boosted regression
Plotly	Interactive application charts
Matplotlib and seaborn	Notebook visualizations
ipykernel	Notebook execution environment

Environment setup

pyproject.toml defines the dependencies and supported Python version. uv.lock records the resolved environment.

Run this command from the project root:

```
uv sync --locked
```

3. Project Structure

```
red_wine_quality/
├── app/ (Interface, data preparation, and evaluation.)
│   ├── __init__.py
│   ├── app.py
│   ├── data.py
│   ├── evaluation.py
│   ├── predict.py
│   └── README.md
├── data/ (Original CSV and cleaned reference copy.)
│   ├── original/
│   │   └── winequality-red.csv
│   └── processed/
│       └── winequality-red-clean.csv
├── notebooks/ (EDA and model-training notebooks.)
│   ├── eda.ipynb
│   └── model_training.ipynb
├── documentation/ (Technical-documentation exports and screenshots.)
│   └── TechnicalDocumentation.pdf
├── paper/ (Final IMRaD DOCX and PDF.)
│   └── FormalDocumentation.pdf
├── .python-version
├── pyproject.toml
├── uv.lock
└── README.md
```

4. Installation and Local Launch

1. Open the project

Download or clone the repository, then open a terminal in the project root—the folder containing pyproject.toml and uv.lock.

2. Install the environment

Install the dependencies using the versions recorded in uv.lock.

```
uv sync --locked
```

3. Start the application

```
uv run streamlit run app/app.py
```

4. Open the application

Open the local URL printed in the terminal. On the first load, the app trains the three models and performs five-fold validation. Allow this process to finish before using the interface.

Later interactions reuse cached results. Restarting Streamlit can trigger training again.

5. Stop the application

Press Ctrl+C in the terminal when you have finished.

5. Data and Input Specification

5.1 Dataset Overview

Source file: data/original/winequality-red.csv

Original data: 1,599 records and 12 columns.

Cleaned data: 1,359 records after removing missing and exact duplicate rows.

Training data is rebuilt from the original CSV. The processed CSV is a cleaned reference copy.

5.2 Prediction Inputs

Use the 11 sliders in the Prediction tab to enter one wine. Quality is the output, not a user input.

The increment is the amount a slider changes with each step.

Input feature	Slider increment
Fixed acidity	0.1
Volatile acidity	0.01
Citric acid	0.01
Residual sugar	0.1
Chlorides	0.001
Free sulfur dioxide	1.0
Total sulfur dioxide	1.0
Density	0.00001
pH	0.01
Sulphates	0.01
Alcohol	0.1

5.3 Input settings and usage

Allowed ranges: Derived from the cleaned dataset and rounded to the slider precision. These are dataset-based operating ranges, not universal chemical validity limits.

Defaults: Feature medians aligned to the slider increments.

Units: Enter measurements in the source dataset's units. A verified units guide still needs to be added.

Supported input: One wine at a time. CSV upload and batch prediction are not implemented.

Validation: Sliders constrain interface inputs. A separate validation interface for arbitrary external inputs is not implemented.

6. Application User Guide

Use the model selector and the three application tabs to explore the data, compare performance, and predict a wine's quality.

6.1 Select a model

Choose Random Forest, XGBoost, or Support Vector Regression from the Model for prediction. The selected model is used when you submit the prediction form.

6.2 Exploration tab

Purpose: Understand the original dataset and its relationships.

- Quality distribution: Compare the number of wines at each score.
- Feature histograms: Inspect measurement ranges and distribution shapes.
- Correlation heatmap: Review linear associations among inputs and quality.
- Boxplots: Compare the three features most correlated with quality across score groups.
- Scatter plot: Explore alcohol and volatile acidity together, colored by quality.

Reading note: These plots use original data; training uses cleaned rows. The scatter plot samples up to 800 records. Associations do not establish causation.

6.3 Model Comparison tab

Purpose: Compare all three models using the same five training-data validation folds. Each metric is shown as a mean and sample standard deviation (SD).

- MAE: Lower is better. Average absolute error in quality points; the primary model-selection metric.
- RMSE: Lower is better. Gives greater weight to large prediction errors.
- R^2: Higher is better. Measures squared-error performance relative to the evaluation-data mean; it is not percentage accuracy.

Reading note: SD describes variation across folds. It is not a confidence interval or an uncertainty range for one wine. These are validation results, not final test results.

6.4 Prediction tab

1. Select the model from Model for prediction.
2. Open Prediction and adjust the 11 chemical-measurement sliders.
3. Select Predict quality to submit the form.
4. Read Estimated quality and Predicted quality score.

Understanding the result

- Estimated quality: The fractional prediction in original quality points, after reversing the training target transform.
- Predicted quality score: The estimate rounded to an integer and clipped to the displayed 3–8 range.
- Mean cross-validation MAE: The selected model's average validation error. It does not measure the error of the particular wine you entered.
- Update your prediction: Results reflect the last submitted values. Submit again after changing the sliders or switching models.

7. Processing and Model Implementation

7.1 Data preparation

1. Load the original CSV and remove missing and exact duplicate rows.
2. Extract the 11 chemical measurements and transform quality using log1p.
3. Create the fixed 80/20 split: 1,087 training rows and 272 held-out rows (random_state=0).

7.2 Model configurations

The following settings are explicitly configured in the current implementation. Other parameters use the installed library's defaults.

Model	Explicit configuration
Random Forest	n_estimators=20

	random_state=0
XGBoost	n_estimators=500; learning_rate=0.05 max_depth=6; subsample=0.8 colsample_bytree=0.8 objective=reg:squarederror; random_state=42
SVR	StandardScaler → SVR C=10; epsilon=0.05 Other settings use library defaults.

7.3 Five-fold evaluation

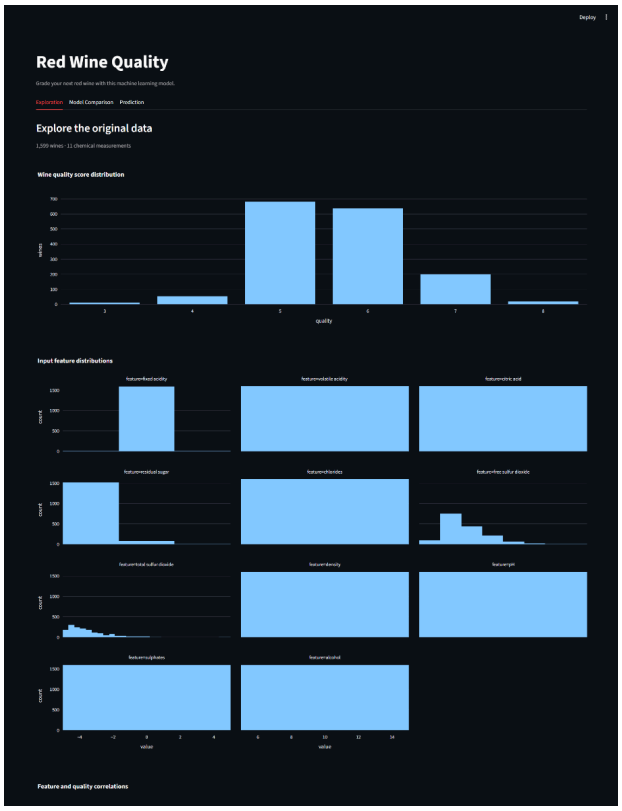
- Shared folds: KFold(n_splits=5, shuffle=True, random_state=0) divides only the training partition. All three algorithms use the same row assignments.
- Fresh fitting: Each fold clones and fits a new model. SVR's scaler learns only from that fold's fitting rows. Models used for app predictions are fitted separately on the full training partition.
- Scoring: Apply expm1 to targets and predictions before calculating MAE, RMSE, and R^2 in original quality points. Report the fold mean and sample SD (ddof=1).
- Current scope: No hyperparameter search is implemented. The app reserves the test partition but does not score it in this workflow. Earlier versions inspected test results; final selection and separate test reporting remain pending.

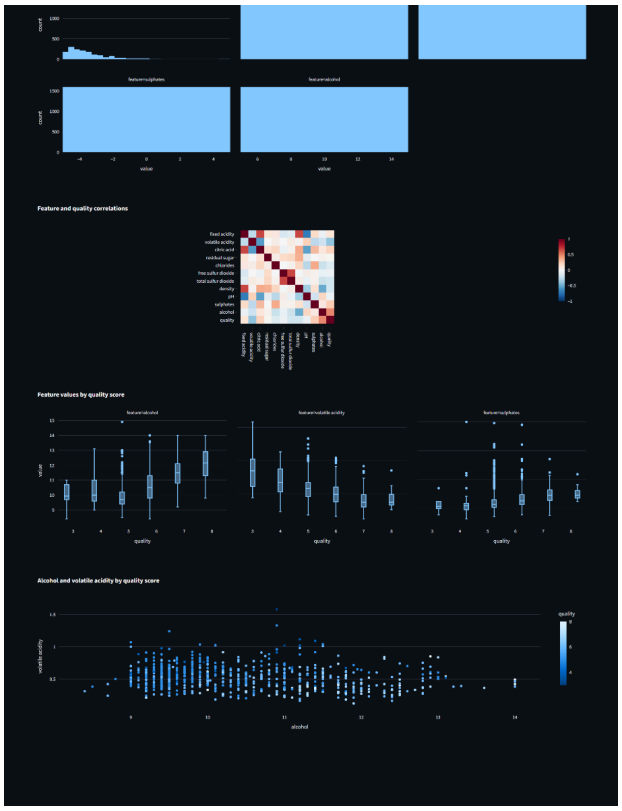
8. Reproducing Notebook Outputs

1. Open your notebook editor and select the project environment's Python kernel.
2. Run all cells in notebooks/eda.ipynb to reproduce the exploratory analysis.
3. Run all cells in notebooks/model_training.ipynb to reproduce training and validation outputs.
4. Save both executed notebooks with their outputs.

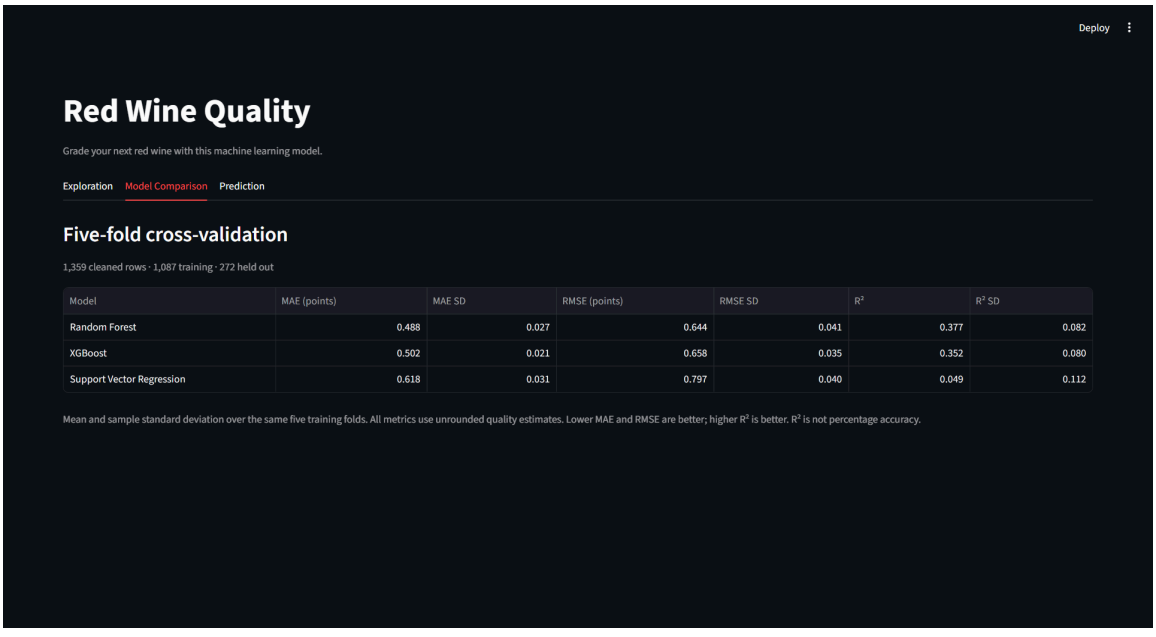
9. Screenshots and Verification Record

Exploration Tab

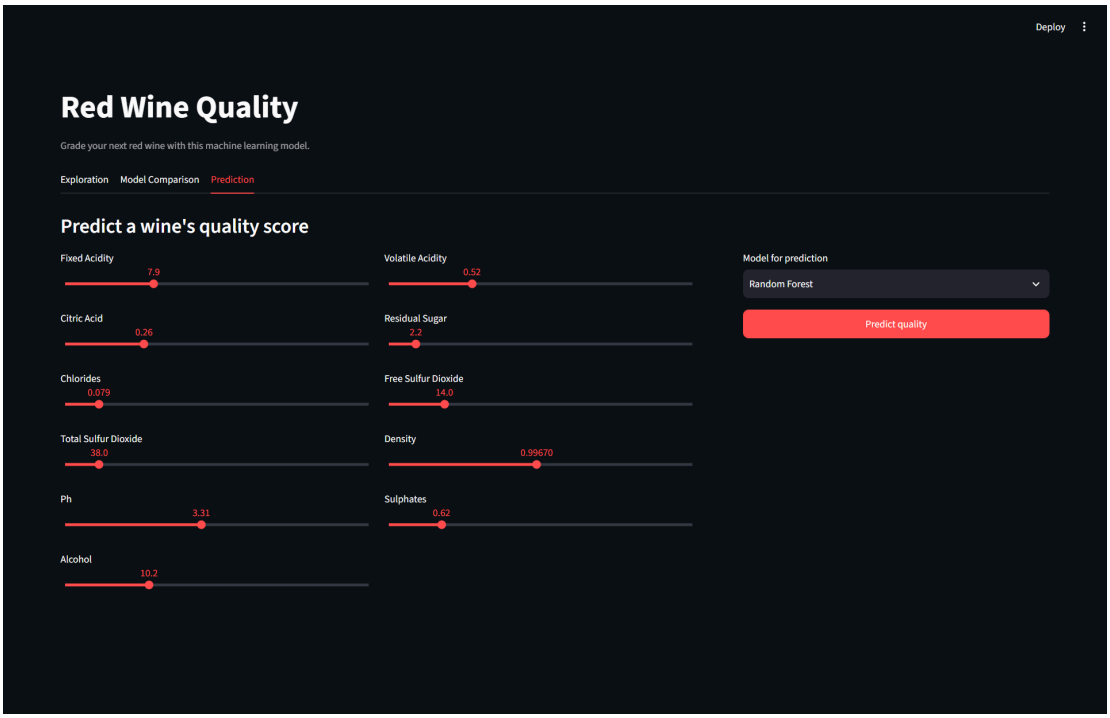




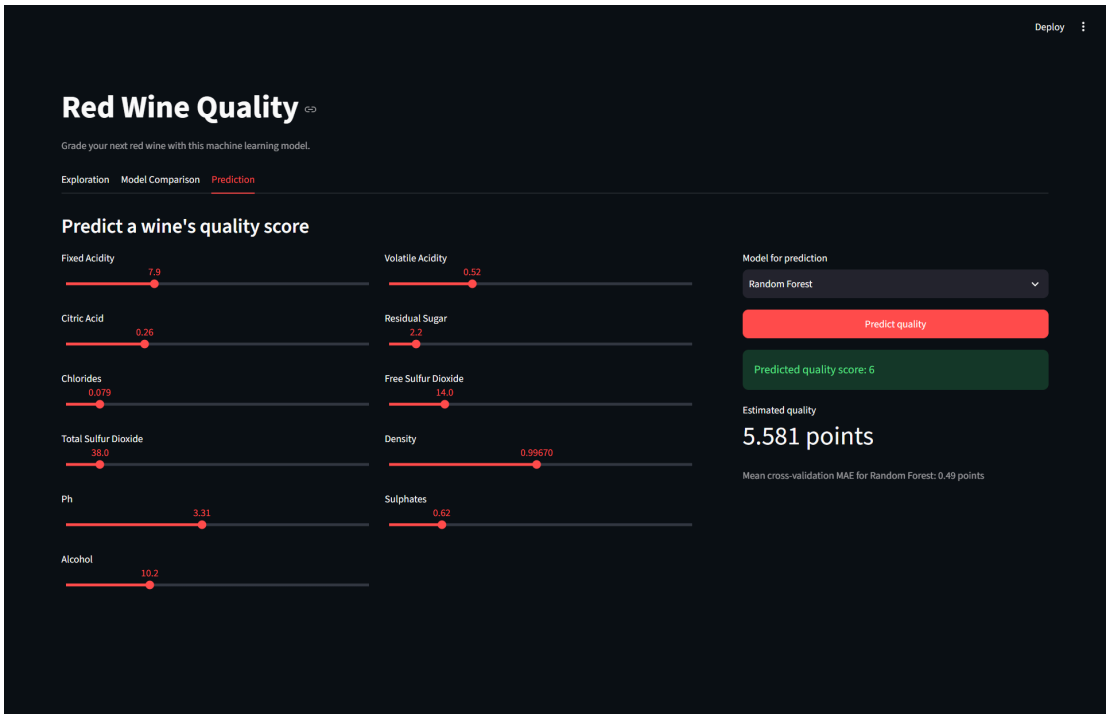
Mode Comparison Tab

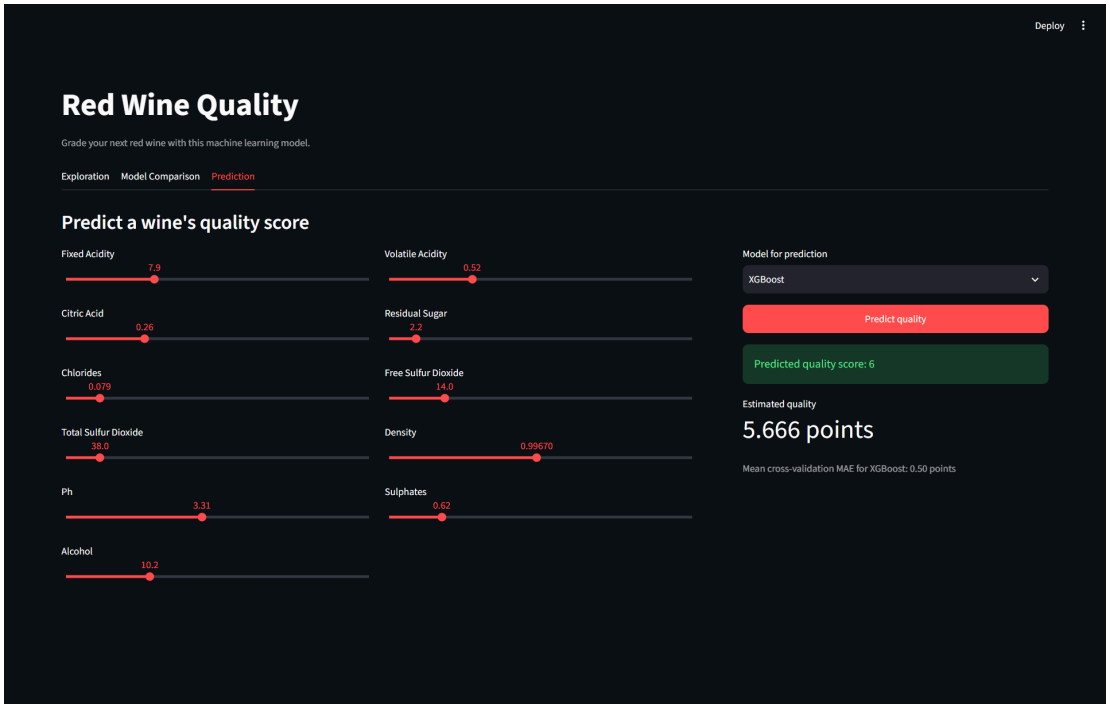


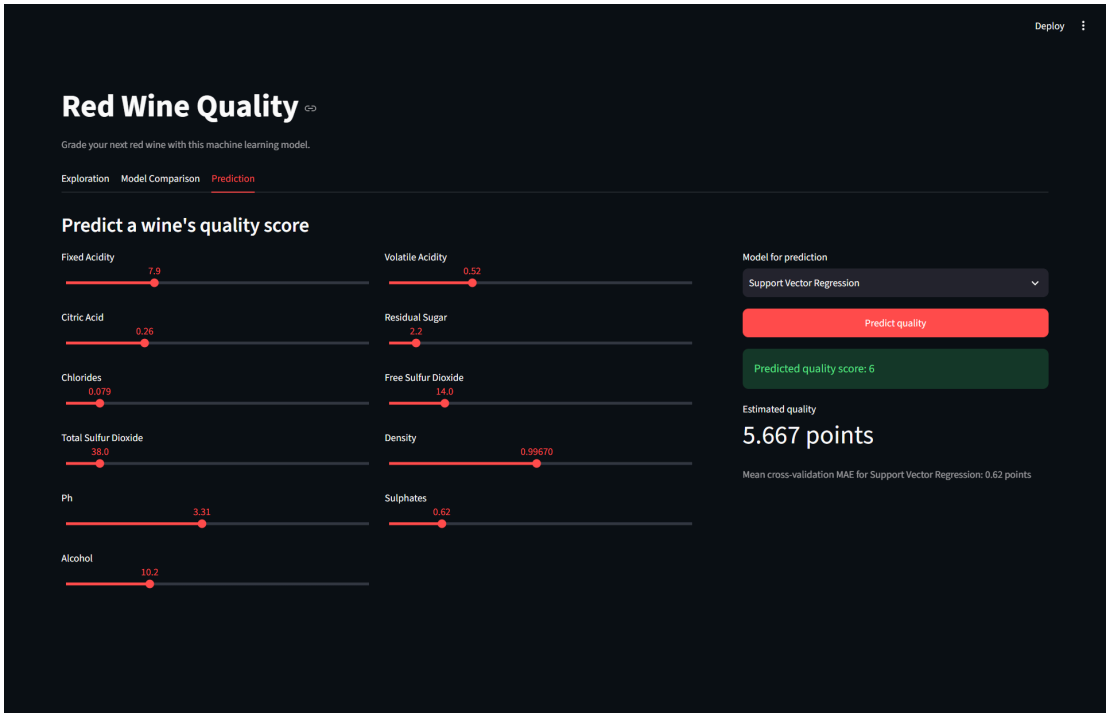
Prediction Tab



Prediction Result




Deploy
Red Wine Quality
Grade your next red wine with this machine learning model.
Exploration
Model Comparison
Prediction
Predict a wine's quality score
Fixed Acidity
7.9
Volatile Acidity
0.52
Citric Acid
0.26
Residual Sugar
2.2
Chlorides
0.079
Free Sulfur Dioxide
14.0
Total Sulfur Dioxide
38.0
Density
0.99670
Ph
3.31
Sulphates
0.62
Alcohol
10.2
Model for prediction
XGBoost
Predict quality
Predicted quality score: 6
Estimated quality
5.666 points
Mean cross-validation MAE for XGBoost: 0.50 points


Deploy
Red Wine Quality
Grade your next red wine with this machine learning model.
Exploration
Model Comparison
Prediction
Predict a wine's quality score
Fixed Acidity
7.9
Volatile Acidity
0.52
Citric Acid
0.26
Residual Sugar
2.2
Chlorides
0.079
Free Sulfur Dioxide
14.0
Total Sulfur Dioxide
38.0
Density
0.99670
Ph
3.31
Sulphates
0.62
Alcohol
10.2
Model for prediction
Support Vector Regression
Predict quality
Predicted quality score: 6
Estimated quality
5.667 points
Mean cross-validation MAE for Support Vector Regression: 0.62 points

10. Troubleshooting

Use the table below to resolve common setup and application issues.

Issue	Recommended action
uv is not recognized	Install uv using its official instructions, then reopen the terminal.
Incompatible Python version	Use Python 3.14 or newer and sync the project environment again.
Missing Python module	Run uv sync --locked. Launch through uv run to use the project environment.
Dataset file not found	Check that data/original/winequality-red.csv exists in the project.
First load takes time	Initial training and 15 validation fits run before results appear. Check the terminal for errors.
Local page does not open	Confirm Streamlit is running and open the local URL printed in the terminal.
Port already in use	Stop the old instance or use: uv run streamlit run app/app.py --server.port 8502
Prediction does not change	Select Predict quality again. Results use the last submitted values.
Results are stale after a CSV change	Restart Streamlit to rebuild cached training and validation results.
Notebook imports fail	Check the selected kernel, working directory, and project-root path setup.

11. Limitations and Remaining Submission Work

11.1 Data and prediction limitations

- Rare scores: Most wines score 5 or 6. Predictions for extreme scores may be less reliable.
- Target assumptions: Quality is an ordinal rating modeled numerically. The benefit of

log1p has not been established through a target-transform comparison.

- Generalization: Results on this dataset do not establish performance on wines from other sources or measurement procedures.

11.2 Application and evaluation scope

- Supported features: The interface predicts one wine at a time. It does not provide an individual confidence interval, batch upload, or external-input API.
- Runtime behavior: Models train at startup and are cached within the running Streamlit process.
- Final evaluation: Selected-model test metrics and residual diagnostics are not included in the current app workflow.

11.3 Before submission

1. Confirm the listed deployment URL works and matches the submitted application.
2. Complete the dataset source, license, original publication, and verified feature units.
3. Add final screenshots and dated clean-install and application-test evidence.
4. Complete the final model evaluation and document its results separately from cross-validation.
5. Export the technical guide as PDF or DOCX into documentation/. Save the separate IMRaD paper DOCX and PDF in paper/.

12. Reference Material

Course requirements

Final Group Project Requirements

Implementation references

- README.md — Project overview and quick-start instructions.
- pyproject.toml and uv.lock — Dependency specification and locked environment.
- app/app.py — Streamlit interface and application workflow.
- app/data.py — Dataset loading, cleaning, target transformation, and split.
- app/evaluation.py — Five-fold validation and metric calculations.

Original Dataset

https://www.kaggle.com/datasets/uciml/red-wine-quality-cortez-et-al-2009